Sanskrit RAG System Report

Submitted by: Aarya Katulwar

1. Introduction

This project implements a CPU-only Retrieval-Augmented Generation (RAG) system for Sanskrit documents. The objective of the project is to build an end-to-end pipeline that can read Sanskrit text documents, retrieve relevant content for a user query, and provide an answer based on the retrieved context.

The project was designed to be simple, modular, and lightweight so that it can run on CPU without requiring GPU support.

2. Objective

The main objective of this assignment is to build a Sanskrit question-answering system that:

- loads Sanskrit documents
- preprocesses and chunks them
- generates embeddings for semantic retrieval
- stores them in a FAISS vector database
- retrieves the most relevant chunks for a query
- returns an answer using retrieved context
- runs entirely on CPU

3. System Overview

The system follows a standard RAG pipeline:

1. Document Loading
2. Text Preprocessing

3. Chunking

4. Embedding Creation

5. FAISS Indexing

6. Query Retrieval

7. Answer Generation / Extractive Fallback

This design was chosen because it is modular, explainable, and suitable for a beginner-friendly implementation.

4. Dataset

The dataset consists of Sanskrit text stories stored as `.txt` files in the `data/` folder.

Each story is stored separately as an individual file.

The dataset used in this implementation includes:

- story1.txt

- story2.txt

- story3.txt

- story4.txt

- story5.txt

These files contain Sanskrit narrative passages used for retrieval and question-answering.

5. Preprocessing

The preprocessing step was implemented in `preprocess.py`.

The following preprocessing steps were applied:

- removal of extra spaces

- removal of repeated blank lines

- preservation of Sanskrit text content

- safe UTF-8 reading

This preprocessing was intentionally kept minimal so that Sanskrit text structure would not be damaged.

6. Chunking Strategy

The cleaned documents were split into smaller chunks using `RecursiveCharacterTextSplitter`.

Chunking configuration

- Chunk size: 500
- Chunk overlap: 80

Reason for this choice

Chunking is necessary because retrieval works better on smaller semantically meaningful units than on full documents. Overlap helps preserve continuity between adjacent chunks.

Very small chunks such as title-only fragments were ignored to improve retrieval quality.

7. Embedding Model

The embedding model used was:

`sentence-transformers/paraphrase-multilingual-MiniLM-L12-v2`

Reason for selection

This model was selected because:

- it supports multilingual text
- it is lightweight
- it works on CPU
- it is suitable for semantic similarity retrieval

The embedding model converts each chunk into a vector representation, which is then stored in FAISS.

8. Vector Database

FAISS was used as the vector database.

Why FAISS was used

- lightweight and efficient
- runs locally
- CPU-friendly
- suitable for semantic search over document chunks

The FAISS index was built in `build_index.py` and stored in:

outputs/faiss_index/

9. Retrieval

The retrieval step was implemented in `query_system.py`.

For each user query:

- the query is embedded using the same embedding model
- FAISS retrieves the top relevant chunks
- the top-k value used was 3

Retrieval quality

Retrieval quality was generally good for the small Sanskrit dataset.

In multiple test cases, the top retrieved chunk correctly matched the relevant source story.

Example successful retrievals:

- sugar-related query retrieved from story1

- bell-related query retrieved from story3

- devotion/help query retrieved from story4

10. Answer Generation

Initially, lightweight generation was tested using `google/flan-t5-small`, but the output quality for Sanskrit context was weak or empty.

The generator was then changed to:

`google/mt5-small`

This improved multilingual compatibility, but generation quality was still inconsistent.

Because of these limitations, the final system used:

- generator model attempt first

- extractive fallback if generation was weak or unusable

11. Extractive Fallback

A simple extractive fallback mechanism was implemented in `generator.py`.

How it works

- the retrieved context is split into sentence-like units

- sentence overlap with the question is calculated

- weak and question-like sentences are penalized

- the most relevant sentence is returned as the fallback answer

Why it was added

This was added because lightweight CPU-friendly generator models did not consistently produce reliable Sanskrit answers.

This fallback improved robustness and ensured the system returned a non-empty answer in most cases.

12. Final Pipeline

The full application was implemented in `app.py`.

The final workflow is:

1. Load retriever
2. Load generator
3. Accept user query
4. Retrieve top relevant chunks
5. Build context from retrieved chunks
6. Generate answer or use extractive fallback
7. Display final answer

This provides a complete end-to-end CPU-only Sanskrit RAG system.

13. Sample Test Cases

Query 1

`शर्करा कुत्र अस्ति ?`

Result:

- relevant chunk retrieved from story1
- answer returned from retrieved context

Query 2

`घण्टां के वादयन्ति ?`

Result:

- relevant chunks retrieved from story3
- retrieval successful
- answer extraction partially correct but not always perfect

Query 3

`देवः कथं साहाय्यम् करोति ?`

Result:

- relevant chunks retrieved from story4
- final answer was meaningful:

  `यदि वयम् प्रयत्नम् कुर्मः, तर्हि एव देवः साहाय्यम् करोति`

14. Strengths of the System

- fully CPU-based
- modular implementation
- clean project structure
- successful FAISS indexing and retrieval
- multilingual embedding support
- end-to-end working pipeline
- fallback answer mechanism improves reliability

15. Limitations

The current implementation has the following limitations:

1. Lightweight generator models do not always produce strong Sanskrit answers.
2. Some final answers are extractive rather than fully natural generated responses.
3. Retrieval works well, but lower-ranked retrieved chunks may sometimes be less relevant.
4. Transliteration support is not implemented as a separate explicit module.
5. The dataset is small, so evaluation is limited.

16. Future Improvements

Possible future improvements include:

- using stronger Sanskrit-aware or multilingual QA/generation models
- adding reranking after retrieval
- improving transliteration handling
- adding better extractive QA logic
- adding a simple user interface with Streamlit
- supporting larger Sanskrit corpora

17. Conclusion

This project successfully implements a CPU-only Sanskrit RAG pipeline using document preprocessing, chunking, multilingual embeddings, FAISS indexing, retrieval, and answer generation with extractive fallback.

Although lightweight generation quality is limited, the retrieval system performs well and the overall pipeline is functional, modular, and suitable as a practical Sanskrit document question-answering system.